Functional Specification

Project Title:	FootStat		
Name:	Alen Tom Joy	**ID:**	18313576
Name:	Jaime De Vivero Woods	**ID:**	19447494
Date Completed:	10/12/21		

Table of contents

1. Introduction

2. General Description

3. Functional Requirements

4. System Architecture

5. High-Level Design

6. Preliminary Schedule

7. Appendices

1. Introduction

1.1 Overview

Our idea is to develop an Android app which takes publicly available statistics on major European soccer leagues from sports websites and displays them in various formats and layouts. The functions available in the app will be to compare teams, view results and the performance of teams in past games, and view the rankings of the top teams and players according to their result after applying various algorithms to help determine their relative strength.

1.3 Glossary

Define and technical terms used in this document. *Only include those with which the reader may not be familiar.*

2. General Description

2.1 Product / System Functions

The product will provide functions to view the latest statistics on major European soccer leagues (Premier League (England), La Liga (Spain), Bundesliga (Germany), Ligue One (France) , Serie A (Italy), Eredivisie(Netherlands), Primeria Liga (Portugal), Austrian Bundesliga), and view past results and standings. It will also offer functions to compare teams and players based on a ranking system which we will implement using different elo algorithms. We will also provide rankings of teams and players using this elo algorithm.

2.2 User Characteristics and Objectives

We plan to make an application that is easy to use, with a simple UI, so we do not expect users to have an expertise in software systems and the application domain. The requirements from the user's perspective is to provide an app which will not only give the latest standings and results, but that will provide a ranking system to give users data that they can use to bet what teams might win competitions or what players might win what trophies.

2.3 Operational Scenarios

Use Cases:

USE CASE 1	**Check team standings for a specific league**
Goal in Context	**User checks team standings**
Scope & Level	**System, Core.**
Preconditions	**User is viewing a specific league**
Success End Condition	**League standings are shown**
Failed End Condition	**League standings are not shown**
Primary, **Secondary Actors**	**User** **System**

Trigger	**User clicks on standings function in the league's options**	
DESCRIPTION	**Step**	**Action**
	1	**User clicks on standings function from the league's page**
	2	**Team standings for that league are checked**
	3	**System updates standings**
	4	**System returns standings**
EXTENSIONS	**Step**	**Branching Action**

USE CASE 2	**Check player rankings**

Goal in Context	**User checks player rankings**	
Scope & Level	**System, Core.**	
Preconditions	**User is in main page**	
Success End Condition	**Player rankings are shown**	
Failed End Condition	**Player rankings are not shown**	
Primary, **Secondary Actors**	**User** **System**	
Trigger	**User clicks on player rankings function in the league's options**	
DESCRIPTION	**Step**	**Action**
	1	**User clicks on player rankings from the main page**
	2	**Player rankings are checked**
	3	**System updates rankings**
	4	**System returns rankings**

EXTENSIONS	**Step**	**Branching Action**

USE CASE 3	**Two teams are compared**
Goal in Context	**User selects two teams to compare**
Scope & Level	**System, Core.**
Preconditions	**User has selected two teams to compare**
Success End Condition	**Team statistics are shown for both teams to compare**
Failed End Condition	**Team statistics are not shown for both teams to compare**
Primary, **Secondary Actors**	**User** **System**

Trigger	**User clicks on a team, and uses the compare function to select another team to compare with**	
DESCRIPTION	**Step**	**Action**
	1	**User clicks on a team**
	2	**User clicks on the compare functions**
	3	**User clicks on a team to compare with**
	4	**System checks stats for both teams**
	5	**System returns stats for both teams**
EXTENSIONS	**Step**	**Branching Action**

2.4 Constraints.

For the design elements, we plan to follow the guidelines set out by Google from the website https://material.io/design. This will allow us to build an app that users will find consistent to the Android platform. We will use the resources at https://developer.android.com/quality to adhere to speed, battery life, functionality, stability and privacy requirements that will enable users to have as flawless an experience as possible. To ensure good database access speed, integrity and logic, we will use the testing tools available for SQLite. The hardware platforms we will support include Android phones and tablets.

3. Functional Requirements

1. Use Retrofit API to collect interpret JSON data from sports data sources / APIs

The system should be capable of retrieving JSON data from source websites or APIs(e.g. Yahoo Sports) and formatting that data into a human readable format to be then stored in SQLite databases locally on the device.

Criticality: Essential - Required for any basic functionality of the application

Technical issues - Creating a performant and battery efficient method of interpreting the JSON into useful data. Retrieving data from multiple sources.

Dependencies: Almost every other requirement is dependent on this.

2. Storing data in local SQLite databases

After the data is formatted, it should be stored in an appropriate manner with the correct relations and logic in an SQLite database on the device. An example of relations that would be required is a player and their team, a team and the trophies for which they compete, teams and scores, etc.

Criticality: Essential - Required for any basic functionality of the application

Technical issues - Ensuring database consistency, access performance and reducing battery drain. Automating the process of inserting data and updating as required whilst maintaining integrity.

Dependencies: Dependent on requirement 1. Almost every other requirement is dependent on this.

3. Create multiple user selectable views of the database

Have several built-in views available by default, e.g. a view to display all the teams competing for certain cups. Also allow the user to define and filter their own views. E.g. Select two teams and compare their performance in the same league.

Criticality: Integral - A major feature of the application integral to its use

Technical issues - Again, ensuring database consistency and integrity, ensuring no illegal options can be selected to prevent null values, loops and crashes.

Dependencies: Dependent on requirements 1 and 2.

4. Apply ELO and Glicko RD rating algorithms to players and teams

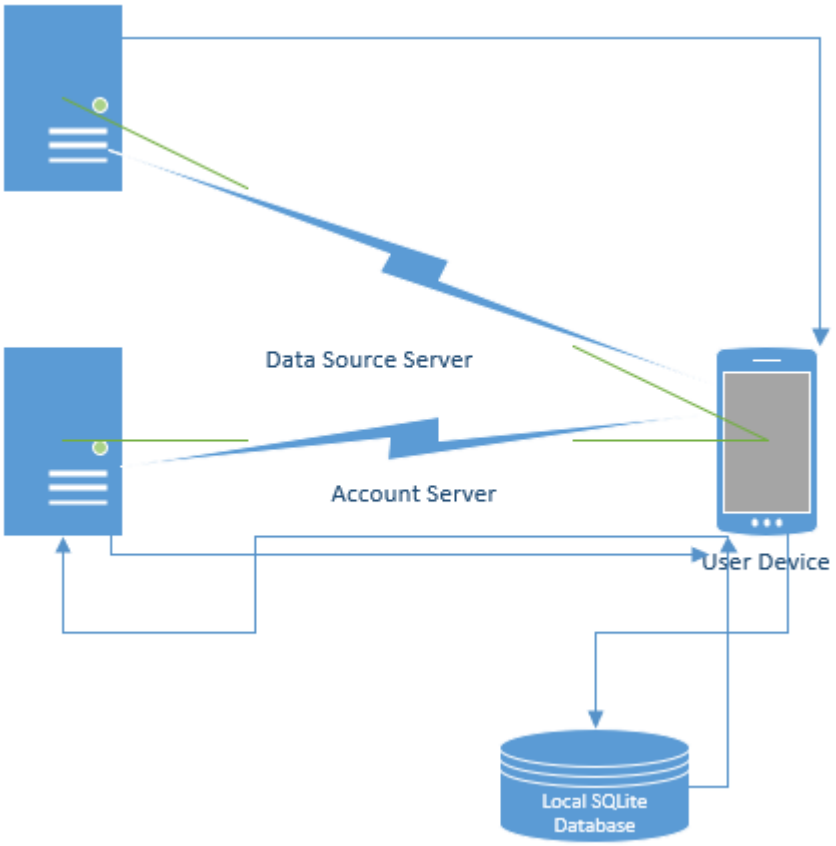
Use the ELO rating system and apply appropriately to players and teams to approximate performance in comparison to other players/teams. This system is already widely used in games such as chess and is well established to be an accurate system when presented with enough data. The Glicko RD rating system is used to determine the accuracy of an ELO rating, and is dependent on the number of games played by a team or player. The more games played, the lower the RD (rating deviation), indicating a higher ELO accuracy.

Criticality: Integral - A major feature of the application integral to its use

Technical issues - Finding the appropriate data for the newness of a team or player. Adjusting the algorithm automatically to fit a standard distribution curve.

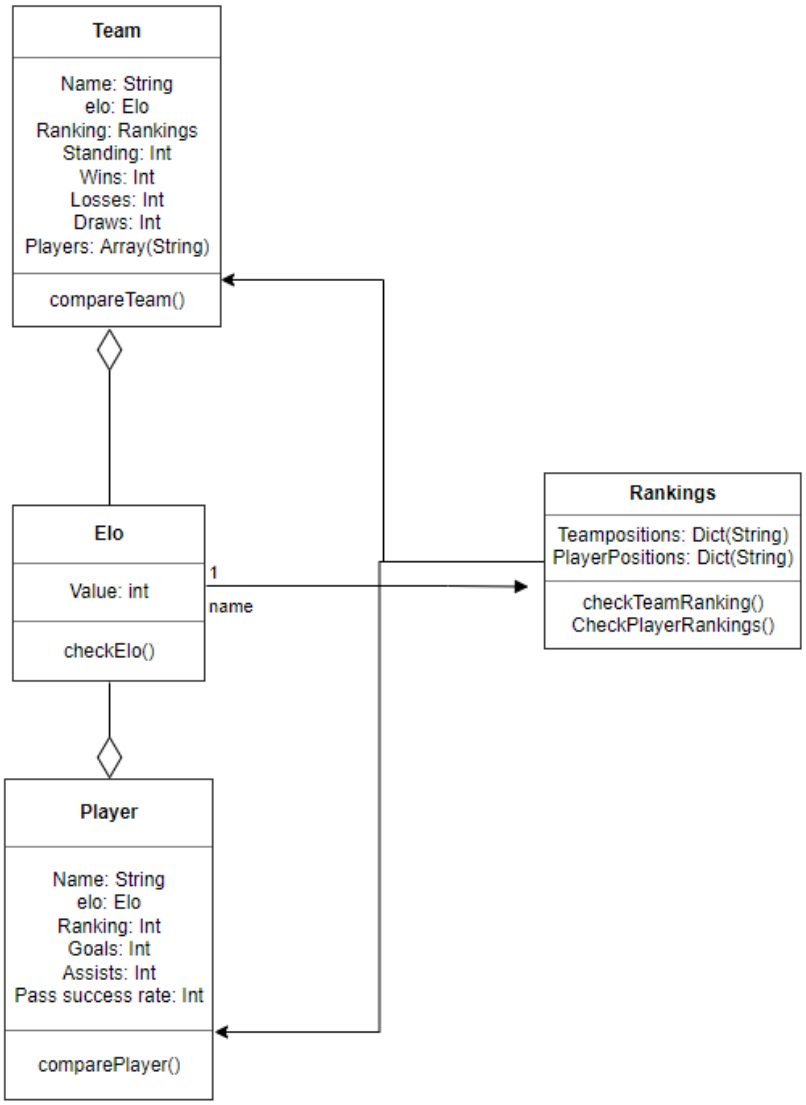
Dependencies: Dependent on requirements 1 and 2.

4. System Architecture

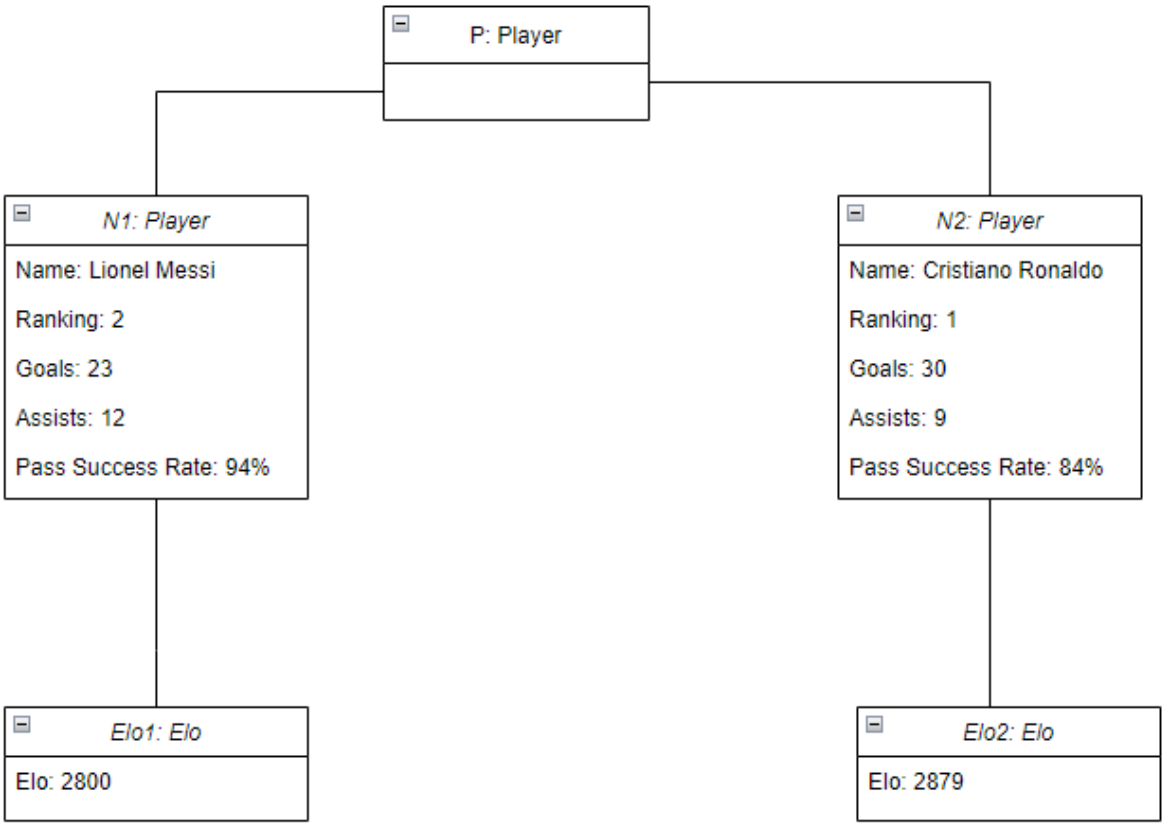


5. High-Level Design

Class Diagram:



Object Models:


P: Player
N1: Player
Name: Lionel Messi
Ranking: 2
Goals: 23
Assists: 12
Pass Success Rate: 94%
N2: Player
Name: Cristiano Ronaldo
Ranking: 1
Goals: 30
Assists: 9
Pass Success Rate: 84%
Elo1: Elo
Elo: 2800
Elo2: Elo
Elo: 2879

T: Team
N1: Team
Name: Manchester United
Ranking: 18
Standing: 7
Wins: 20
Losses: 12
Draws: 6
Players: (De Gea,..., Sancho)
E1: Elo
Value: 2400

6. Preliminary Schedule

Preliminary software/hardware requirements:

Languages: Java, Kotlin, SQL, JavaScript

Tools: MySQL Workbench, Adobe XD, IntelliJ Idea, Android Studio, Webstorm, Socket.io, Retrofit API, Websockets, Git

The major tasks and their brief descriptions are outlined below:

1. Interpreting JSON data - due before 31/12/21

   Consolidate sources from which to retrieve JSON data, and create a method of interpreting the data. Complete testing on this.

2. Store data in SQLite database and create default views - due 14/01/22

   Create a consistent and performant database with the interpreted JSON data with minimal errors and null values. Add relations to allow for the selection of default database views. Complete testing.

3. Enable user defined views and create user server - due 31/01/22

   Allow users limited functionality over choosing their own views and filters as defined in functional requirements. Add required relations for this and make sure no illegal operations are possible. Create a heroku server to process user accounts. Complete testing.

4. Implement rating algorithms - due 10/02/22

   Add in the rating algorithms with appropriate relations in the database and test.

5. Begin UI design process and integrate working features - continuously integrated due 24/02/22

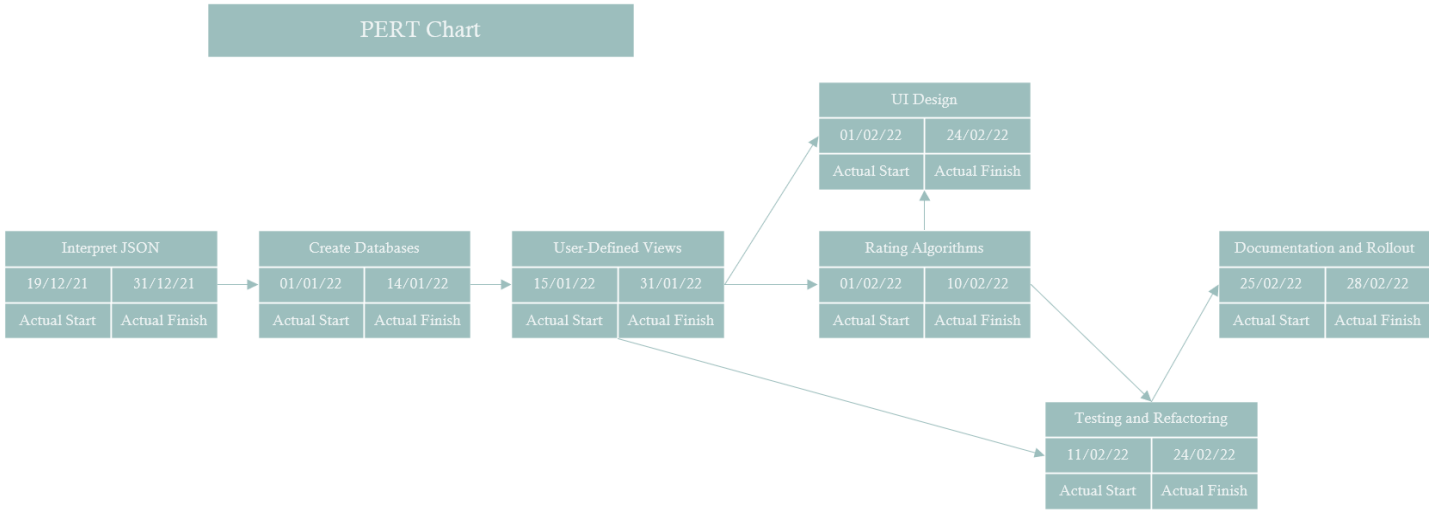
   Design and implement the UI, following the principles set out in the Google design documentation for features that have passed tests and are working.

6. Refactor code and finalise testing - due 24/02/22

   Refactor code as necessary to maximise performance, increase battery life, reduce storage, memory and time complexity. Perform as much testing as possible in this stage.

7. Complete documentation and app rollout on Play Store due 28/02/22

Finish all required documentation, manuals, terms and conditions, GDPR and data processing as required, citations and license usage.



7. Appendices

SQLite testing:

https://sqlite.org/testing.html